

1. What are the main types of Dependency Injection?

- **Constructor Injection:** Dependencies are provided through a class constructor (preferred for mandatory dependencies).
- **Setter/Method Injection:** Dependencies are assigned via setter methods (good for optional dependencies).
- **Field/Property Injection:** Dependencies are assigned directly to fields, often using annotations (e.g., `@Autowired` in Spring).

2. What is the difference between Transient, Scoped, and Singleton lifetimes?

These define how long a dependency lives in a container:

- **Transient:** A new instance is created every time it is requested.
- **Scoped:** A new instance is created once per client request (e.g., within one HTTP request).
- **Singleton:** A single instance is created once and reused throughout the application's lifetime.

3. Why is Constructor Injection generally preferred over Setter Injection?

“Constructor injection is preferred because it ensures the object is always in a valid, fully initialized state, promotes immutability, and makes dependencies mandatory. Setter injection can result in a partially initialized object.”

4. What is a Dependency Injection Container?

“A DI container (or IoC container) is a framework that automates the instantiation, configuration, and management of object dependencies. It automatically resolves and injects required dependencies into classes, reducing boilerplate code.”

5. How does Dependency Injection improve unit testing?

“By using DI, you can easily replace real, complex implementations (e.g., a live database connection) with mock or stub objects during testing. This allows for testing components in isolation, improving test speed and reliability.”

6. What is the difference between DI and the Service Locator pattern?

“In DI, the dependency is provided to the class, making dependencies explicit (via constructors). With a Service Locator, the class actively asks the locator for its dependency, hiding the dependency behind a static call and making it harder to track.”

7. What happens if a service is not registered in the DI container?

“The application will typically throw an exception—often `InvalidOperationException` or a similar "service not found" error—at runtime when trying to resolve that service, because the container does not know how to construct it.”

8. What is the purpose of Generational Garbage Collection?

“It divides the heap into young and old generations, assuming most objects die young. Minor GC clears the Young generation frequently, while Major/Full GC clears the Old generation, reducing overall pause times.”

9. How does an object become eligible for GC?

“An object becomes eligible when it is no longer reachable, typically by nullifying its reference variable (`obj = null`), reassigning it, or creating it inside a method that has finished execution.”

10. What is "Stop-the-World" (STW) in GC?

STW is a phase where all application threads are paused to allow the garbage collector to run safely, which can impact application performance.

11. Can you force garbage collection to run?

No, calling `System.gc()` or `Runtime.getRuntime().gc()` only suggests that the JVM run the garbage collector; it cannot force it.

12. What is the difference between Minor and Major GC?

Minor GC cleans the young generation (fast, frequent), while Major GC cleans the old generation/tenured space (slower, less frequent).

13: What are some common JVM GC algorithms?

Common algorithms include Serial GC, Parallel GC, CMS (Concurrent Mark & Sweep), and G1 (Garbage First).

14. Why do I need to override `hashCode()` if I only override `equals()` to check if two Employee objects have the same ID?

Scenario1:Custom Objects in a HashSet (Need to Override Both): Without overriding `hashCode()`, two Employee objects with the same ID will have different hash codes (defaulting to memory address). HashSet will place them in different "buckets," failing to detect them as duplicates, thus breaking the set contract.

```
class Employee {  
  
    int id;  
  
    String name;  
  
    // Overriding equals() to check only ID
```

```

public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Employee emp = (Employee) o;
    return id == emp.id;
}

// Correctly overriding hashCode() based on ID
public int hashCode() {
    return Objects.hash(id);
}
}

```

15. Scenario: Hash Collisions (Same Hash, Different Content) - If two different objects have the same hashCode(), does HashMap treat them as equal?

No. If hashCode() is the same, HashMap uses equals() to verify actual equality. If equals() returns false, they are treated as distinct objects in the same bucket (linked list).

Example:

- String objects "Aa" and "BB" may produce the same hashCode() in some JVM implementations.
- "Aa".hashCode() == "BB".hashCode() $\rightarrow$ true (Collision)
- "Aa".equals("BB") $\rightarrow$ false (Different content)

16. Scenario: Using Only equals() (Without hashCode()) - What happens if I only override equals()?

The objects will be treated as equal when using == or .equals(), but hash-based collections (HashMap, HashSet) will not work correctly. For instance, myMap.get(newKey) will return null even if a "logically" equal key exists.

17. Scenario: Immutable Fields in hashCode() - Can I use mutable fields in hashCode()?

It is dangerous. If an object is added to a HashSet, and then a field used in hashCode() is changed, the object's bucket changes, and HashSet will never find it again, creating a memory leak/data inconsistency.

18. What are Lambda Expressions and their main advantage?

Lambda expressions are anonymous functions (methods without names, return types, or modifier) that allow treating functionality as a method argument or code as data.

Advantage: They provide a concise way to represent a functional interface and reduce code, especially when using the Collections framework.

Example: (a, b) -> a + b;

19. What is the Functional Interface and @FunctionalInterface annotation?

A functional interface is an interface containing exactly one abstract method (Single Abstract Method - SAM).

Annotation: While not mandatory, @FunctionalInterface ensures the interface has only one abstract method, causing a compiler error if more are added. Examples include Runnable, Callable, and the new Predicate, Consumer, and Supplier interfaces.

20. What is the Purpose of the Stream API?

The Stream API is a functional-style, declarative approach to processing sequences of elements (e.g., collections).

Key Operations: filter (for filtering), map (for transformation), sorted (for sorting), and collect (to get result). Streams are lazy and do not store data.

21. What are Default Methods in Interfaces?

Introduced to support backward compatibility, default methods allow adding new methods to interfaces with a default implementation without breaking existing implementation classes. They are defined using the default keyword.

Example:

```
interface Interface {  
    default void newMethod() {  
        System.out.println("Default implementation");  
    }  
}
```

22. What is the Optional Class?

Optional<T> is a container object used to represent a value that may or may not exist (i.e., it can be null).

Usage: It helps avoid NullPointerException and promotes safer, more readable code without explicit null checks.

Example: Optional.ofNullable(value).orElse("Default");

23. How do Method References differ from Lambda Expressions?

Method references (:: syntax) are a more readable shorthand for lambda expressions that simply call an existing method.

Example: Instead of `str -> System.out.println(str)`, you can use `System.out::println`.

24. What is the Difference between `map()` and `flatMap()`?

`map()` is used for transformation, mapping each element to a new value. `flatMap()` is used for flattening, taking a stream of streams and flattening it into a single stream.

Example: `map()` converts a `Stream<String>` to `Stream<Integer>` (lengths), while `flatMap()` can convert a `Stream<List<String>>` to a `Stream<String>`.

25. What is the String constant pool, and where is it located in the Java memory model?

The String constant pool is a special memory area in Java where the JVM stores literal strings and strings created using `String.intern()`. Its purpose is to optimize memory usage by storing only one copy of each unique string value.

In the original Java memory model (prior to Java 7), the String constant pool was part of the permanent generation (PermGen) space. However, in Java 7 and later versions, it was moved to the heap space, where it is more dynamically managed. This move helps avoid `OutOfMemoryError` issues that could occur in the fixed-size PermGen space.

26. Explain the difference in memory allocation for `String s1 = "abc";` versus `String s2 = new String("abc");`

The two methods handle string allocation differently:

- **`String s1 = "abc";`**: This uses a string literal. The JVM first checks the String constant pool for the value "abc". If it already exists, `s1` is made to point to the existing object in the pool. If it does not exist, a new string object is created in the pool and `s1` points to it.
- **`String s2 = new String("abc");`**: This forces the creation of a new string object on the heap, regardless of whether "abc" already exists in the constant pool. An additional string literal "abc" is typically also created in the constant pool during compilation/runtime if not already present, resulting in two objects in memory (one on the heap, one in the pool).

27. Are strings mutable in Java, and how does the immutability impact memory allocation?

No, strings are immutable in Java. This means that once a String object is created, its content cannot be changed. Any operation that appears to modify a string, such as concatenation (e.g., `s = s + "xyz";`) or using methods like `replace()`, actually creates an entirely new string object in memory with the modified content. The original string remains unchanged.

28. What is an Exception and how does it differ from an Error?

An exception is an event occurring during program execution that disrupts the normal flow, usually recoverable (e.g., `FileNotFoundException`). An error represents a serious system-level issue (e.g., `OutOfMemoryError`) that an application generally cannot recover from.

29. What is the difference between throw, throws, and throwable in Java?

throw: Used to explicitly raise an exception within a method.

throws: Declares an exception in the method signature, indicating the method might throw this error.

Throwable: The parent class of all exceptions and errors in Java.

30. Does the finally block always get executed?

The finally block is designed to execute regardless of whether an exception is thrown or caught, making it ideal for cleanup activities like closing files or database connections. However, it will not execute if the JVM crashes or `System.exit()` is called.

31. What is Exception Propagation?

If an exception occurs in a method and is not caught, it propagates (moves up) the call stack to the calling method, repeating until caught or the program terminates.

32. What is the purpose of try-with-resources?

Answer: Introduced in Java 7, try-with-resources ensures that each resource (such as streams or connections) is closed at the end of the statement, preventing resource leaks and reducing the need for explicit finally blocks.

33. How do you create a Custom Exception?

By creating a new class that inherits from the base `Exception` class (or one of its subclasses), allowing developers to define application-specific error conditions.

34. What is type casting in Java, and what are its two main types?

Type casting is the process of converting a value from one data type to another. It is necessary when you need to use a value of one type where a different type is expected.

The two main types are:

Widening Casting (Implicit/Automatic): Converts a smaller data type to a larger one. This is done automatically by the compiler because there is no risk of data loss (e.g., `int` to `long`).

Narrowing Casting (Explicit/Manual): Converts a larger data type to a smaller one. This must be done manually using the cast operator () and can potentially lead to data loss (e.g., double to int).

35: Provide an example of both widening and narrowing casting for primitive data types.

Widening Casting Example:

```
int number = 10;

long largeNumber = number; // Automatic/implicit casting

System.out.println(number); // Output: 10

System.out.println(largeNumber); // Output: 10
```

Narrowing Casting Example:

```
double decimalValue = 9.78;

int intValue = (int) decimalValue; // Manual/explicit casting

System.out.println(decimalValue); // Output: 9.78

System.out.println(intValue); // Output: 9 (decimal part is lost/truncated)
```

36. What is a ClassCastException?

A ClassCastException is a runtime error that occurs when you try to cast an object reference to a type that is not compatible with its actual underlying type. This typically happens during downcasting in object-oriented programming if the object is not an instance of the target subclass.

37. How does type casting work with inheritance (upcasting and downcasting)?

Upcasting: This is the automatic conversion of a subclass reference to a superclass reference. It is safe because a subclass object is always an instance of its superclass. The reference can only access methods defined in the superclass.

```
class Animal { public void sound() { System.out.println("Animal makes a sound"); } }

class Dog extends Animal { public void sound() { System.out.println("Dog barks"); } }

Animal animal = new Dog(); // Upcasting (implicit)

animal.sound(); // Output: Dog barks (calls the overridden method in Dog)
```

Downcasting: This is the conversion of a superclass reference to a subclass reference. It must be done explicitly using the cast operator () and requires the use of the instanceof operator to ensure safety and prevent a ClassCastException at runtime.

```
Animal animal = new Dog();
```

```
if (animal instanceof Dog) {  
    Dog dog = (Dog) animal; // Downcasting (explicit)  
    // You can now access Dog-specific methods if any  
}
```

38. How are collisions handled in a HashMap?

Java's HashMap uses chaining to handle collisions. When multiple keys hash to the same bucket, they are stored in a linked list or a balanced tree (in newer Java versions).

39. Can you use a custom object as a key?

Yes, but the custom class must implement equals() and hashCode() methods. The hash code should remain consistent after the object is inserted.

40. Explain the difference between == and equals () method ?

Both == and the equals() method are used for comparison, but they operate differently: == compares references (memory addresses), while equals() compares the content or value of objects.

41. What are some design tactics we can use while developing, that can help your class to be more testable?

Key design tactics for testable classes include:

- [Dependency Injection](#): Pass dependencies (e.g., databases, services) through constructors or setters rather than instantiating them inside the class. This allows swapping real objects for mocks during tests.
- Single Responsibility Principle (SRP): Keep classes small and focused on one function. Smaller, focused classes are easier to isolate and test.
- Interface Segregation: Depend on interfaces rather than concrete implementations, making it easier to swap components.
- Avoiding Singletons: Avoid global state, which makes tests dependent on order and difficult to run in parallel.
- Boundary Value Analysis: Test the edges of input ranges, not just the middle.
- Deterministic Logic: Ensure that the same input always produces the same output, eliminating flakiness.
- Minimize Logic in View/UI: Move business logic to separate testable classes (e.g., presenters, view models).

42. Questions on Collections.

Array vs ArrayList: Arrays are fixed-size and fixed-type, whereas ArrayLists are dynamic, resizable, and only hold objects.

ArrayList vs LinkedList: ArrayList uses a dynamic array (faster search, $O(1)$). LinkedList uses a doubly-linked list (faster insertion/deletion, $O(1)$).

HashSet vs TreeSet: HashSet is backed by a HashMap and is unsorted. TreeSet is backed by a TreeMap and maintains elements in sorted order.

43. HashMap Internal Working:

HashMap works on the principle of hashing, using hashCode() to find a bucket and equals() to find the key, allowing $O(1)$ average time complexity for operations.

44. Iterator vs. ListIterator:

Iterator allows traversing in one direction (forward) and removing elements. ListIterator allows bidirectional traversal and adding/modifying elements.

45. Comparable vs. Comparator: Comparable (java.lang) sorts objects based on natural ordering (compareTo method). Comparator (java.util) sorts objects based on customized ordering (compare method).

Fail-Fast vs. Fail-Safe: Fail-Fast iterators (like ArrayList) throw ConcurrentModificationException if the collection is modified while iterating. Fail-Safe (like ConcurrentHashMap) operate on a clone and do not throw exceptions.

46. What is a PriorityQueue?

A queue that processes elements based on priority (natural ordering or a comparator) rather than FIFO (First-In-First-Out).

47. Difference between HashMap and ConcurrentHashMap?

“HashMap, Hashtable, and ConcurrentHashMap in Java are all map implementations, but they differ in thread-safety and performance. HashMap is non-synchronized and faster for single-threaded tasks. Hashtable is synchronized, thread-safe, but slow due to locking the entire table. Concurrent HashMap provides the best performance for concurrent, multi-threaded scenarios by locking only portions (buckets/segments) of the map.”

- **HashMap:** Not thread-safe and faster for single-threaded tasks, allows one null key and multiple null values, fast performance.
- **ConcurrentHashMap:** High performance thread-safe implementation. It locks only specific segments, allowing multiple threads to read and write concurrently. It does not allow `null` keys or values, and its iterators do not throw `ConcurrentModificationException`.

48. What is the difference between a process and a thread?

A process is an executing instance of a program, while a thread is a subset of a process. Threads within the same process share memory, making communication faster, whereas processes have separate memory spaces.

49. What is a race condition?

A race condition occurs when multiple threads attempt to modify shared data simultaneously, leading to unexpected results. This is resolved by using synchronization mechanisms.

50. What is the ThreadLocal class?

ThreadLocal provides separate, isolated copies of variables for each thread, preventing shared state issues.

51. How do you achieve thread safety in Java?

Safety can be achieved using synchronization (methods/blocks), volatile variables, atomic classes (AtomicInteger), and thread-safe collections (ConcurrentHashMap).

52. What is a deadlock and how do you avoid it?

A deadlock occurs when multiple threads wait on each other to release resources, creating a circular wait. It is avoided by acquiring locks in a consistent order, using timeouts (tryLock), and minimizing locked sections.

53. What is a daemon thread?

A user thread is a high-priority task, while a daemon thread is a low-priority background thread (e.g., garbage collector) that stops when all user threads finish.

54. Can you start a thread twice?

No, you cannot restart a finished thread. Calling start() on a thread that has already run throws an IllegalStateException.

55. What is the purpose of start() vs run() method?

The start() method creates a new, separate thread of execution and then calls the run() method in that new thread. If you call run() directly, it executes in the current thread, defeating the purpose of multithreading.

Resources and References:

Concept	Resource Url
Strings related questions	https://www.baeldung.com/java-string-interview-questions

DSA or Java related exercises	https://www.geeksforgeeks.org/java
----------------------------------	---